

Part 4

Toolchain, Tests, CI and Operations

Everything around the code - the Docker/Polygeist image and its patches, GitHub workflows and the current CI state, the complete test suite anatomy, fixtures, reports and baselines, papers and legacy tooling, and runbooks for day-to-day work.

AUDIENCE

Anyone setting up a machine, running or writing tests, touching CI or the Docker image, producing results, or debugging an environment problem.

COVERS

Docker image walkthrough, toolchain.lock, GHCR publishing and export, CI workflow, test suite by file, fixtures directories, report schemas and baselines, research and papers folders, legacy tools, runbooks, troubleshooting

REPOSITORY STATE

branch main lineage, commit d1ab454

GENERATED

2026-09-20

SOURCE

docs/handbook/source/04-toolchain-tests-ci.md

Contents

The Docker Toolchain Image	3
Why it exists	3
docker/polygeist/Dockerfile step by step	3
toolchain.lock	4
Image names and how LoopHole finds them	4
Publishing and exporting	4
Legacy tooling (tools/legacy/)	4
Continuous Integration	5
.github/workflows/ci.yml	5
Current state	5
The Test Suite	6
Running tests	6
conftest.py	6
Unit tests (tests/unit/, 15 files)	7
Integration tests (tests/integration/, 13 files)	8
Fixture directories (tests/fixtures/)	9
Writing a good test in this codebase	9
Reports, Baselines, Research and Papers	10
Report formats	10
Committed baselines (research/benchmarks/baselines/)	10
research/	10
papers/	11
Runbooks	12
Set up a new machine	12
Lift a C kernel	12
Investigate a surprising result	12
Produce a batch report	13
Rebuild the Docker image after a toolchain change	13
Regenerate the handbook	13
Troubleshooting	14

The Docker Toolchain Image

Why it exists

Polygeist needs a matching LLVM, Clang and MLIR build, which historically took hours to compile and was hard on Windows. The April 2026 migration moved everything into a Docker image so that the host only needs Python. The history is in [docs/planning/docker-polygeist-migration-plan.md](#) and [docs/status/docker-polygeist-delivery-handoff.md](#); superseded approaches are in [tools/legacy/](#).

docker/polygeist/Dockerfile step by step

The build context is `packages/loophole` (only `requirements.txt` is copied from it). Build arguments:

`LLVM_VERSION=17`, `LLVM_GITHUB_TAG=llvmorg-17.0.6`, `POLYGEIST_REF=llvm17bump`, `BUILD_THREADS=4`.

1. **Base and packages.** `ubuntu:24.04`; apt installs build tools, `clang-17`, `libclang-17-dev`, `libmlir-17-dev`, `llvm-17-dev`, `llvm-17-tools`, `mlir-17-tools`, Ninja and Python. `PATH` and `LD_LIBRARY_PATH` point at `/usr/lib/llvm-17`.
2. **Sources.** Shallow clone of Polygeist branch `llvm17bump`. A sparse, blob-less clone of `llvm-project` at `llvmorg-17.0.6` containing only Clang internals Polygeist needs (`clang/include`, `clang/lib/CodeGen`, `clang/lib/Driver`, `clang/tools/driver`).
3. **Patch 1: drop GPU passes.** Removes `ConvertParallelToGPU.cpp`, `SerializeToCubin.cpp`, `ParallelLoopDistribute.cpp`, `ConvertPolygeistToLLVM.cpp` and the `MLIRGPUOps/MLIRNVMDialect` link dependencies from `lib/polygeist/Passes/CMakeLists.txt`, deletes the files, and asserts the tokens are gone.
4. **Patch 2:** removes `MLIRGPUOps` from `tools/cgeist/CMakeLists.txt` (asserted).
5. **Patch 3:** adds `#include <set>` to `lib/polygeist/Ops.cpp` (raises if the insertion point is missing).
6. **Patch 4: driver.cc.** Guards the CUDA toolchain include, adds the `ControlFlow` dialect include and loads the dialect, removes GPU-cubin pass registration and the `-opaque-pointers=0` argument, stubs out the `CPUify` pass, and removes the Polygeist-to-LLVM lowering pass. None of these replacements are asserted: if upstream text changes, they silently do nothing. The practical consequence of this patch set is that this `cgeist` can emit MLIR but cannot lower programs to LLVM, which is fine for LoopHole.
7. **Patch 5: clang-mlir.cc** API updates for LLVM 17 (`memcpy volatile` flag, `opaque pointers`, `OpenMP::IsTargetDevice`, `getLLVMLinkageVarDefinition`, `APIInt::getAllOnes`); three tokens are asserted. `CGCall.cc` `memmove/memset/memcpy` signatures, `CGStmt.cc` `llvm::None`, and `TypeUtils.cc` `opaque` pointer replacements are silent.
8. **Build.** CMake with Ninja against the apt LLVM/MLIR/Clang CMake packages, target `cgeist` only, `-j BUILD_THREADS`.
9. **Install.** Copies `cgeist` to `/usr/local/bin`; `cgeist++` is copied if it exists, otherwise a symlink to `cgeist`.
10. **Python.** `pip install --break-system-packages -r requirements.txt` (LoopHole itself is not installed). `WORKDIR /workspace`, `CMD /bin/bash`.

The resulting image is about 3.4 GB. First build takes about five minutes on 12 CPUs and 8 GB of RAM; lower `BUILD_THREADS` if the compiler is killed for lack of memory.

toolchain.lock

```
LLVM_TAG=llvmorg-17.0.6
LLVM_REF=d10fdd5422ed6abf9ee9a874dbe5165fbdeb47d0
POLYGEIST_BRANCH=llvm17bump
POLYGEIST_REF=4e125563267267aa341d94dabd7178534d64ca6e
```

The lock file is documentation only

The Dockerfile never reads it. Polygeist is cloned at the branch head, LLVM comes from Ubuntu packages, and the base image is not pinned by digest. Rebuilding the image months apart can therefore produce a different `cgeist`. Part 5, F-31 proposes making the Dockerfile consume the lock.

Image names and how LoopHole finds them

Two tags are expected locally: `loophole-polygeist:llvm17` and

`ghcr.io/schizoid-man/loophole-polygeist:llvm17`. `SETUP.md` builds both because different code paths default to different names:

Code path	Default image
<code>loophole compile</code>	<code>ghcr.io/schizoid-man/loophole-polygeist:llvm17</code> (fixed default)
<code>loophole cgeist, lift-c</code>	first existing of the two tags, overridable by flag or env var
MLIR verifier	first existing of the two tags, overridable by env var
PowerShell report wrappers	GHCR name

Publishing and exporting

`.github/workflows/publish-docker-image.yml` runs on manual dispatch or on pushes to `main` that touch `docker/polygeist/**`, `packages/loophole/requirements.txt` or the workflow itself. It builds with Buildx using `BUILD_THREADS=2` and pushes `ghcr.io/<owner>/loophole-polygeist` with tags `llvm17`, `latest` and `sha-<short>`. The last run (2026-09-13) succeeded in under five minutes.

`.github/workflows/export-docker-image-artifact.yml` runs only on manual dispatch: pulls an existing tag, `docker save | gzip`, splits into 1.9 GB parts if needed with reassembly instructions, and uploads as a workflow artifact with configurable retention. Collaborators without a GitHub token for the private package download the artifact and `docker load` it.

Legacy tooling (`tools/legacy/`)

Folder	What it was	Status
<code>docker-source-build/</code>	Dockerfile, entrypoint and smoke test that compile LLVM, MLIR, Clang and Polygeist from source (multi-hour)	Unsupported; paths refer to the pre-restructure layout
<code>wsl-native-build/</code>	WSL2 scripts and <code>INSTALL.txt</code> for a native build, plus a PowerShell launcher	Unsupported; superseded by Docker

Continuous Integration

`.github/workflows/ci.yml`

Triggers: pushes to `main` or `master`, and every pull request. One job on `ubuntu-latest`, working directory `packages/loophole`, Python 3.11.

Step	What it does
Install Python dependencies	<code>pip install -r requirements.txt</code> then <code>pip install -e .</code>
Install MLIR verifier tool	<code>apt-get install mlir-tools</code> or <code>mlir-18-tools</code> or <code>mlir-17-tools</code>
Resolve verifier binary	Exports <code>LOOPHOLE_MLIR_VERIFY_CMD</code> for the first of <code>mlir-opt</code> , <code>mlir-opt-18</code> , <code>mlir-opt-17</code> on PATH
Run tests with artifact validation required	<code>pytest tests/ -q</code> with <code>LOOPHOLE_REQUIRE_MLIR_VERIFY=1</code>
Trusted lane batch smoke check	<code>python -m loophole.cli batch tests/fixtures --profile trusted --include-glob matmul.mlir --max-files 1 --no-write-emitted</code>
Trusted lane CI guard checks	Re-runs three named tests from <code>test_cli_batch_report.py</code>
B-16 reliability guard checks	Re-runs three named parser/emitter tests

Note that CI validates artifacts with whatever `mlir-opt` version Ubuntu provides (not necessarily 17), while local runs use the Docker image's 17.0.6. The last two steps re-run tests already executed in the full run; they add time, not coverage.

Current state

CI on main is failing

From the restructure commit (run 34763839671, 2026-09-13) until LOOPH-10, every push to `main` failed with `1 failed, 364 passed (test_lift_larger_matmul)`: Z3 recursive-function names collided between candidate checks (F-06). LOOPH-10 fixed it.

Other observations: GitHub warns that `actions/checkout@v4` and `actions/setup-python@v5` target the deprecated Node 20 runtime; there is no lint, type check or coverage step; only one Python version is tested.

The Test Suite

Running tests

From `packages/loophole` with the virtual environment active:

```
python -m pytest tests -q                # everything (expect 392 passed)
python -m pytest tests/unit -q           # unit tests only
python -m pytest tests/integration/test_matmul.py -q
python -m pytest tests -q -k "stablehlo" # by keyword
python -m pytest tests -q --cov=loophole  # coverage (needs the [dev] extra)
```

Tests that validate emitted artifacts use the `mlir_verifier_cmd` fixture: they use a real verifier if one is found (Docker image or `mlir-opt` on PATH), skip if only the internal fake exists, and fail instead of skipping when `LOOPHOLE_REQUIRE_MLIR_VERIFY=1` or the environment profile is `ci-strict/trusted`. With Docker running, a full local run takes about ten seconds.

`conftest.py`

Located at `packages/loophole/conftest.py` (package root, not inside `tests/`). Provides session-scoped `affine_extractor`, `z3_checker` (15 s timeout), `linalg_emitter`, `stablehlo_emitter`, `lifter_linalg` and `lifter_stablehlo` (from the resolved profile), `policy_profile_name`; function-scoped MLIR text fixtures for eight kernels; session-scoped parsed `LoopNestInfo` fixtures for five kernels; and `mlir_verifier_cmd`. A helper `load_mlir_fixture(name)` reads files from `tests/fixtures/`.

Unit tests (`tests/unit/`, 15 files)

File	Tests	What it covers	Quality notes
<code>test_affine_extractor.py</code>	49	Loop counts, bounds, IV roles, reads/writes, mixed affine/SCF, index normalisation, stress fixtures, dynamic and unranked memrefs, zero dimensions	Positive cases only; no mutants
<code>test_emitter.py</code>	42	Linalg and StableHLO text for many sketches, metadata gates, transpose permutations, conv attribute inference, three real artifact validations	Mostly substring assertions; see F-08
<code>test_a14_counterexample_replay.py</code>	33	Binding parsing, expression evaluator, source/sketch simulation, report fields	Thorough for a small module
<code>test_a15_shape_parametric_proof.py</code>	26	Parametric report fields, <code>_verify_parametric</code> , lifter parametric mode	Accepts <code>ENCODE_ERROR</code> as a valid outcome; asserts on private methods
<code>test_sketch_library.py</code>	22	Library integrity and presence of key sketches	Structural only
<code>test_a13_multi_reduction_boundary.py</code>	16	Reduction pattern classification, <code>ENCODE_ERROR</code> notes for uninferable forms, <code>check</code> never raises	Tests private classification
<code>test_a16_proof_quality_summary.py</code>	15	Grade thresholds, confidence distribution, per-sketch breakdown, Markdown rendering	Tests script internals via <code>sys.path</code> import
<code>test_z3_checker.py</code>	29	Structural match (incl. scale/negate/max/min rules), basic equivalence and non-equivalence, report fields, expression parsing, symbolic dot, RecFunction isolation, sketch/tensor rank mismatch, unmodelled op combinations	Few negative proofs here; mutant kernels live in <code>test_elementwise_lift.py</code>
<code>test_parser_diagnostics.py</code>	11	Each diagnostic producer	Good
<code>test_lifter_semantics.py</code>	10	State mapping, partial success, strict acceptance, profile resolution, no-verdict error message	Good contract tests
<code>test_polygeist_frontend.py</code>	8	Command building, stdout/file output, errors, Docker fallback	Fully mocked subprocess
<code>test_parser_unsupported_form_diagnostics.py</code>	4	Diagnostics for <code>iter_args</code> , conditionals, no loop, no store	Good
<code>test_parser_corpus_compiler.py</code>	3	Corpus counts and classes	Couples to exact corpus contents
<code>test_weekly_benchmark_report.py</code>	2	Fixture discovery, trend and canonical summaries	
<code>test_fixture_source_policy.py</code>	2	Greps two test files for inline MLIR	Policy enforced by text search

Integration tests (`tests/integration/`, 13 files)

File	Tests	What it covers	Quality notes
<code>test_matmul.py</code>	17	Full pipeline for matmul variants, StableHLO strict path, missing shapes, artifact validation, mixed loops, dynamic dims, <code>iter_args</code> form	Several <code>if result.success:</code> vacuous assertions; mixed step-2 fixture asserted as success
<code>test_cli_strict_policy.py</code>	16	Strict/no-verify rejection, exit codes per profile for <code>lift</code> , <code>lift-c</code> , <code>demo</code>	Exit codes checked against mocked lift results
<code>test_conv2d.py</code>	14	Conv2d simple and NHWC, attribute inference, artifact validation, 1x1 and 5x5 proofs	
<code>test_cli_batch_report.py</code>	13	Report fields, profiles, selection filters, output layout, trusted-lane validation	Trusted-lane tests mock both lift and verifier discovery
<code>test_cli_error_messages.py</code>	11	Error and warning text for malformed input	
<code>test_stablehlo_phase3_ops.py</code>	10	StableHLO transpose, add, subtract, multiply, vecdot, matvec, conv1d, conv2d, reduce sum, reduce max	No artifact validation possible (F-09)
<code>test_elementwise_lift.py</code>	10	Elementwise kernels proved as the op they compute (mul, relu); mutant kernels (div, negate, scale by constant, add constant, three inputs, mul-add, hex constant) never PROVED	Exact sketch and state asserted
<code>test_transpose.py</code>	8	2-D and non-square transpose, permutation, artifact validation	
<code>test_conv1d.py</code>	6	Conv1d identification and attribute inference	Includes <code>test_result_has_input_mlir</code> that is conditional
<code>test_dot.py</code>	6	Dot products including symbolic N and disagreement metadata	
<code>test_matvec.py</code>	4	Matvec square, tall, wide	
<code>test_weekly_benchmark_report.py</code>	3	Report generation and writing	
<code>test_demo_script_smoke.py</code>	2	<code>examples/run_demo.py</code> plain and demo modes with a fixture limit	

Total: 392 tests (unit 272, integration 120).

Fixture directories (`tests/fixtures/`)

Path	Contents	Used by
<code>*.mlir</code> (matmul, transpose, conv1d, conv2d, dot_product, elementwise_add, reduce_sum)	Hand-written kernels mirroring the Python strings	<code>batch</code> smoke test, weekly script, CI trusted smoke
<code>*_lifted.mlir</code>	Committed outputs from earlier batch runs, some StableHLO and some Linalg	Nothing reads them; overwritten when <code>batch</code> runs on the directory
<code>a12_weak_kernels/</code>	conv1d, conv2d, dot, matvec (square, tall, wide), reduce_sum and lifted outputs	A-12 baseline generation
<code>a15_parametric/</code>	matmul, matvec, row reduce_sum and lifted outputs	A-15 pilot
<code>corpus/</code>	Polygeist-style inputs: affine matmul (supported), affine.apply indices, conv1d SCF, dot with iter_args, if-guarded store, vectorised no-loop; <code>parser_compat_summary.json/.md</code>	Corpus compatibility tests; excluded from <code>batch</code> by default
<code>loophole_report.json</code>	A committed batch report	Nothing reads it

Two parallel fixture sources exist (Python strings and `.mlir` files) with overlapping content and different names.

Writing a good test in this codebase

1. For a bug, write the failing test first and mark it `@pytest.mark.xfail(strict=True, reason="fixed by T<n>")`; the fixing PR removes the marker.
2. Assert the exact expected sketch and state; never `if result.success:`.
3. For every new "proves X" test, add a mutant that must not prove X.
4. Validate emitted MLIR with the real verifier for its dialect.
5. Test public behaviour (`Lifter.lift`, CLI commands) rather than private methods where practical, and avoid mocking the component under test.
6. Name tests by behaviour, not by plan task ID.

Reports, Baselines, Research and Papers

Report formats

Artifact	Producer	Schema	Key contents
Batch report <code>loophole_report.json</code>	<code>loophole batch --report</code>	1.1 (compatible 1.0)	run metadata, selection, summary counts, canonical StableHLO summary, 25 slowest, per-file results with SHA-256
Weekly benchmark <code>weekly_benchmark_<label>_<ts>.json/.md</code>	<code>scripts/generate_weekly_benchmark_report.py</code>	1.2 (compatible 1.0, 1.1)	summary, trend deltas and anomalies, canonical summary, proof-quality summary, per-kernel rows
Proof-quality artifact <code>proof_quality_<label>_<ts>.json/.md</code>	<code>scripts/generate_proof_quality_summary.py</code>	1.0	proof-quality summary and optional trend against a baseline
Corpus compatibility	<code>scripts/analyze_parser_corpus_compat.py</code>	none	totals, class frequency, per-file rows

The weekly and proof-quality scripts write to `packages/loophole/reports/benchmarks/`, which is git-ignored. Results that were meant to be kept were copied by hand into `research/benchmarks/baselines/`.

Committed baselines (`research/benchmarks/baselines/`)

File	Date	What it records
<code>sprint1_week1_linalg_20260417.json</code> , <code>sprint1_week1_stablehlo_20260417.json</code>	2026-04-17	A-12 week-1 weak-kernel baseline
<code>a12_week2_weakkernels_linalg_20260421.json</code> , <code>..._stablehlo_20260421.json</code>	2026-04-21	A-12 completion
<code>a13_a14_a15_sprint3_baseline_20260426.json</code>	2026-04-26	23 kernels, 18 proved, 5 refuted
<code>proof_quality_a16_pq_20260429_20260429T110512Z.json/.md</code>	2026-04-29	Grade B, 78.3% strict acceptance

Historical records, not evidence of correctness

These files contain absolute Windows paths from the original machines (`POC_initial_demo\tests\fixtures\...`) and verdicts from the unsound checker (Part 5, F-01, F-12, F-28). Keep them as history; do not use them as regression baselines until regenerated after Phase 1.

research/

`experiments/` contains only `_template/README.md` (hypothesis, setup with commit SHA and image tag, commands, results, conclusion); `notebooks/` is empty apart from a README. `CONTRIBUTING.md` defines the reproducibility rule: any published number must be traceable to a commit, an image tag, the exact command and the raw output.

`papers/`

`thesis-report/` (LaTeX chapters, logos, committed `main.pdf`) and `ieee-paper/` (`main.tex`, `main.pdf`, and `docs/implementation_findings.md`, described as the factual source for the paper). The IEEE abstract is appropriately cautious ("not a claim of a completed production lifter"), but its evidence section cites a StableHLO batch report (7 files, 5 proved, 2 refuted) whose JSON file is not in the repository, and its proved/refuted distinctions depend on the checker behaviour described in Part 5. Build with `latexmk -pdf main.tex`; build artifacts are git-ignored.

Runbooks

Set up a new machine

1. Install Docker Desktop (WSL2 backend on Windows), Python 3.11 or newer, and Git. Start Docker.
2. From the repository root, build both tags: `docker build -f docker/polygeist/Dockerfile -t loophole-polygeist:llvm17 -t ghcr.io/schizoid-man/loophole-polygeist:llvm17 packages/loophole.`
3. Verify: `docker run --rm loophole-polygeist:llvm17 bash -lc "which cgeist mlir-opt && mlir-opt --version"` should show LLVM 17.0.6.
4. `cd packages/loophole`, create and activate a venv, `pip install -r requirements.txt` and `pip install -e ".[dev]"`.
5. Windows: set `PYTHONUTF8=1` (per session or with `setx`).
6. Run `python -m pytest tests -q` and confirm the 364/1 baseline.

Lift a C kernel

```
loophole compile path/to/kernel.c -o out/kernel.mlir --print-command
loophole lift out/kernel.mlir --target linalg --report -o out/kernel_linalg.mlir
```

If the C file contains more than one function (for example a `main`), use `loophole lift-c kernel.c --function kernel_name` instead, because the parser merges functions (F-05). Check the emitted MLIR yourself: `docker run --rm -v "$PWD/out:/work" loophole-polygeist:llvm17 mlir-opt /work/kernel_linalg.mlir`.

Investigate a surprising result

1. Rerun with `-v` to see candidates, confidences and each Z3 verdict.
2. Run `loophole verify file.mlir -v` to see every sketch's verdict, including structural mismatches.
3. For a refutation, add `--show-replay`, or run `loophole batch ... --report` and then `loophole replay report.json --entry N`.
4. Inspect the parsed structure directly:

```
python -c "from loophole.affine_extractor import AffineExtractor as E; import sys; i=E().extract(open(sys.argv[1]).read()); print(i.bounds, i.parallel_vars, i.reduction_vars, [(a.tensor_name, a.index_exprs) for a in i.reads+i.writes], [(o.op_type, o.operands) for o in i.compute_ops]); [print(d) for d in i.diagnostics]" file.mlir
```

5. Compare against the known-defect list in Part 5 before assuming a new bug.

Produce a batch report

```
loophole batch tests/fixtures --target linalg --profile exploratory --report --report-path out/report.js
on --no-write-emitted
```

Use `--no-write-emitted` or `-o out/` when running on `tests/fixtures`, otherwise committed `*_lifted.mlir` files are overwritten. Use `--profile trusted` only with Docker running or a real `mlir-opt` on PATH, and remember StableHLO validation always fails today.

Rebuild the Docker image after a toolchain change

1. Edit the Dockerfile and, if refs change, `toolchain.lock` in the same PR.
2. Build locally with `--build-arg BUILD_THREADS=2` if memory is limited, run the verification command and the full test suite.
3. Merge to `main`; the publish workflow pushes `llvm17`, `latest` and `sha-*` tags.
4. Collaborators pull (with a `read:packages` token) or download the export workflow's artifact.

Regenerate the handbook

```
pip install reportlab
python docs/handbook/build_handbook.py          # all parts
python docs/handbook/build_handbook.py 05       # one part
```

Troubleshooting

Symptom	Cause and fix
<code>error from registry: denied</code> when pulling	The GHCR package is private. Build locally, or <code>docker login ghcr.io</code> with a <code>read:packages</code> token, or use the export workflow artifact.
Unable to find image during <code>compile</code>	Only one tag was built; <code>compile</code> defaults to the GHCR name. Tag both names or pass <code>--docker-image</code> .
<code>UnicodeDecodeError: 'charmap' codec on Windows</code>	Set <code>PYTHONUTF8=1</code> .
Input/output/include paths must be on the same drive	Docker mounts one common parent; keep sources and outputs on one drive.
<code>COPY requirements.txt ... not found</code> during build	Wrong build context; the last argument must be <code>packages/loophole</code> .
Build killed with <code>c++: fatal error: Killed</code>	Out of memory: <code>--build-arg BUILD_THREADS=2</code> or more Docker memory.
None of the N checked candidates reached a Z3 verdict.	Every checked candidate hit <code>ENCODE_ERROR</code> or <code>STRUCTURAL_MISMATCH</code> ; the message lists each one's reason. It is not a refutation.
Lift reports <code>PROVED</code> but the MLIR looks wrong	Likely a known soundness defect (F-01 to F-05). Check the kernel against Part 5 chapter 2 and validate with <code>mlir-opt</code> .
<code>Validation failed ... Dialect 'stablehlo' not found</code>	Expected: the image has no StableHLO tools (F-09).
<code>--profile trusted</code> passes on a machine without Docker	The internal fake verifier was used (F-10). Start Docker or install <code>mlir-opt</code> .
<code>loophole lift --no-verify</code> hangs on a big kernel	<code>--no-verify</code> removes the Z3 timeout (F-11). Use <code>--z3-timeout</code> instead.
<code>loophole replay</code> raises <code>KeyError: '-'</code>	The report entry had no Z3 verdict (F-17).
Old virtual environment broken after pulling the restructure	It points at <code>POC_initial_demo/</code> . Delete it and recreate under <code>packages/loophole</code> .
<code>loophole</code> not found or PowerShell blocks <code>Activate.ps1</code>	Activate the venv, or <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> , or call <code>.venv\Scripts\loophole.exe</code> .